multiple sheets in it. We use the *read_excel* function to read the data from it. Pandas defaults to storing data in DataFrames. We then store this DataFrame into a variable.

```
import pandas as pd
df=pd.read_excel('C:/Users/Neethu/product.xlsx')
print(df)
```

```
          Product     Price
0     Computer   24700
1           Tablet     12250
2           iPhone    14000
3           Laptop    25000
```

For reading specific columns and rows, use the following code:

```
import pandas as pd
data = pd.read_excel('C:/Users/Neethu/input.xlsx')

# Use the multi-axes indexing function
print (data.loc[[1,3],['salary','name']])
```

For reading multiple Excel files, type the following code:

```
import pandas as pd
with pd.ExcelFile('C:/Users/Neethu/input.xlsx') as xls:
    df1 = pd.read_excel(xls, 'Sheet1')
    df2 = pd.read_excel(xls, 'Sheet2')


print("****Result Sheet 1****")
print (df1[0:5]['salary'])
print("")
print("***Result Sheet 2****")
print (df2[0:5]['zipcode'])
```

```
****Result Sheet 1****
0    10000
1    20000
2    23456
3    12345
4    23412
Name: salary, dtype: int64

***Result Sheet 2****
0    89765
1    12345
2    23456
3    34567
4    45678
Name: zipcode, dtype: int64
```

Python and relational databases

SQLite, a database included with Python, creates a single file for all the data in a database. It is built into Python but is only built to be accessed by a single connection at a time. SQLite is self-contained, serverless, very fast and lightweight; and the entire database is stored in a single disk file.

SQLite can be integrated with Python using a Python module called *sqlite3*. You do not need to install this module separately because it comes bundled with Python version 2.5.x onwards.

The Python SQL toolkit, SQLAlchemy, which is said to be the best Python database abstraction tool, provides an accessible and intuitive way to query, build and write to SQLite, MySQL and PostgreSQL databases. SQLAlchemy is an open source toolkit, object-relational mapper and an additional library for implementing database connectivity, which provides full SQL language functionality to be used in Python.

To use a database, create a connection object, which will represent the database:

```
import sqlite3
connection = sqlite3.connect("College.db") // created a
database with the name "College"
```

After creating an empty database, add one or more tables to it using the *create table sql* syntax. To send a command to 'SQL' or SQLite , we need to create a cursor object by calling the *cursor()* method of connection, as shown below:

```
cursor = connection.cursor()
```

Now, declare a *create table* with a triple quoted string in Python, as follows:

```
cmd = """CREATE TABLE employee (ssn INTEGER PRIMARY KEY,
fname VARCHAR(20), lname VARCHAR(30), salary number, birth_
date DATE);"""
```

Now we have a database with a table but no data included. To populate the table, execute the *INSERT* command to SQLite.

```
cursor.execute(cmd)


cmd = """INSERT INTO employee VALUES ("12398", "John",
"Danny", "80000", "1961-10-25");"""
```